

Gripper Configuration **gripperControl**

It is used to control the gripper and get its state. The RealMan robotic arm supports various common grippers. In the system, the INSPIRE-ROBOTS EG2-4C2 gripper is adapted for use. To facilitate user operation of the gripper, the robotic arm controller provides an open gripper control protocol (the gripper control protocol is mutually exclusive with the end modbus functionality). The following is a detailed description of the member functions of the gripper configuration **gripperControl**, including the method prototype, parameter description, return value, and usage demo.

Set gripper stroke **rm_set_gripper_route()**

It refers to the maximum and minimum opening values of the gripper. Once set successfully, the values are saved automatically and will not be lost when the gripper is powered off.

- **Method prototype:**

```
rm_set_gripper_route(self, min_route: int, max_route: int) -> int:
```

python

- **Parameter description:**

Parameter	Type	Description
min_route	int	Minimum opening value of the gripper, range: 0-1,000, without a unit of measurement; max_route (int): maximum opening value of the gripper, range: 0-1,000, without a unit of measurement

- **Return value:**

State codes executed by functions:

0 represents success. For other error codes, please refer to the [API2 Error Codes](#).

- Usage demo

```
from Robotic_Arm.rm_robot_interface import *

# Instantiate the RoboticArm class
arm = RoboticArm(rm_thread_mode_e.RM_TRIPLE_MODE_E)

# Create the robotic arm connection and print the connection ID
handle = arm.rm_create_robot_arm("192.168.1.18", 8080)
print(handle.id)

print(arm.rm_set_gripper_route(70, 200))

arm.rm_delete_robot_arm()
```

python

Release gripper `rm_set_gripper_release()`

The gripper moves to the maximum opening position at the specified speed.

- Method prototype:

```
rm_set_gripper_release(self, speed: int, block: bool, timeout: int) -> int:
```

python

- Parameter description:

Parameter	Type	Description
<code>speed</code>	<code>int</code>	Release speed of the gripper, range: 1–1,000, without a unit of measurement.
<code>block</code>	<code>bool</code>	true: blocking mode, where the system waits for the controller to return the gripper in-position command; false: non-blocking mode, where no gripper in-position command is received.
<code>timeout</code>	<code>int</code>	Blocking mode: set the timeout period for waiting for the in-position gripper, in s; non-blocking mode: 0: immediately return after sending the command; other values: return after receiving the successful setting command.

- **Return value:**
State codes executed by functions

Parameter	Type	Description	Handling Suggestions
0	int	Success.	-
1	int	The controller returns false, indicating that the parameters are sent incorrectly or the robotic arm state is wrong.	- Validate JSON Command: ① Enable DEBUG logs for the API to capture the raw JSON data. ② Check JSON syntax: Ensure correct formatting of parentheses, quotes, commas, etc. (You can use a JSON validation tool). ③ Verify against the API documentation that parameter names, data types, and value ranges comply with the specifications. ④ After fixing the issues, resend the command and check if the controller returns a normal status code and business data. - Check Robot Arm Status: ① Check for real-time error messages in the robot arm controller or logs (such as hardware failures, over-limit conditions), and reset, calibrate, or troubleshoot hardware issues according to the prompts. ② After fixing the issues, resend the command and check if the controller returns a normal status code and business data.
-1	int	The data transmission fails, indicating that a problem occurs during the communication.	Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-2	int	The data reception fails, indicating that a problem occurs during the communication, or	- Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal. - Verify Version Compatibility:

Parameter	Type	Description	Handling Suggestions
		the controller has a return timeout.	① Check if the controller firmware version supports the current API functions. For specific version compatibility, refer to the Version Description. ② If the version is too low, upgrade the controller or use an API version that is compatible.
-3	int	The return value parse fails, indicating that the received data format is incorrect or incomplete.	Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version compatibility, refer to the Version Description. ② If the version is too low, upgrade the controller or use an API version that is compatible.
-4	int	Timeout.	- Check Timeout Setting: In blocking mode, it supports configuring the timeout for waiting for the device to complete its motion. Ensure that the timeout is set longer than the device's motion time. - Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-5	int	The current in-position equipment verification fails, indicating the current in-position equipment is not a gripper.	- Detect Concurrent Control by Multiple Devices: Check if other devices are sending motion commands to the robot arm, including the motion of the robot arm, gripper, dexterous hand, and elevator. - Monitor Command Events in Real-Time: Register the callback function <code>rm_get_arm_event_call_back</code> : ① Capture device arrival events (such as motion completion, timeout, etc.). ② Determine the specific device type that

Parameter	Type	Description	Handling Suggestions
			triggered the event through the device parameter in the callback.

- Usage demo

python

```
from Robotic_Arm.rm_robot_interface import *

# Instantiate the RoboticArm class
arm = RoboticArm(rm_thread_mode_e.RM_TRIPLE_MODE_E)

# Create the robotic arm connection and print the connection ID
handle = arm.rm_create_robot_arm("192.168.1.18", 8080)
print(handle.id)

print(arm.rm_set_gripper_release(500, True, 10))

arm.rm_delete_robot_arm()
```

Gripper force-controlled grasping `rm_set_gripper_pick()`

The gripper grasps with the set speed and force. When the gripping force exceeds the preset threshold, the grasping operation stops.

- Method prototype:

python

```
rm_set_gripper_pick(self, speed: int, force: int, block: bool, timeout: int) -> int:
```

- Parameter description:

Parameter	Type	Description
<code>speed</code>	<code>int</code>	Grasping speed of the gripper, range: 1–1,000, without a unit of measurement
<code>force</code>	<code>int</code>	Force control threshold, range: 50–1,000, without a unit of measurement

Parameter	Type	Description
<code>block</code>	<code>bool</code>	true: blocking mode, where the system waits for the controller to return the gripper in-position command; false: non-blocking mode, where no gripper in-position command is received.
<code>timeout</code>	<code>int</code>	Blocking mode: set the timeout period for waiting for the in-position gripper, in s; non-blocking mode: 0: immediately return after sending the command; other values: return after receiving the successful setting command.

- **Return value:**
State codes executed by functions:

Parameter	Type	Description	Handling Suggestions
0	<code>int</code>	Success.	-

Parameter	Type	Description	Handling Suggestions
1	int	The controller returns false, indicating that the parameters are sent incorrectly or the robotic arm state is wrong.	- Validate JSON Command: ① Enable DEBUG logs for the API to capture the raw JSON data. ② Check JSON syntax: Ensure correct formatting of parentheses, quotes, commas, etc. (You can use a JSON validation tool). ③ Verify against the API documentation that parameter names, data types, and value ranges comply with the specifications. ④ After fixing the issues, resend the command and check if the controller returns a normal status code and business data. - Check Robot Arm Status: ① Check for real-time error messages in the robot arm controller or logs (such as hardware failures, over-limit conditions), and reset, calibrate, or troubleshoot hardware issues according to the prompts. ② After fixing the issues, resend the command and check if the controller returns a normal status code and business data.
-1	int	The data transmission fails, indicating that a problem occurs during the communication.	Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-2	int	The data reception fails, indicating that a problem occurs during the communication, or the controller has a return timeout.	- Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal. - Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version compatibility, refer to the Version Description.

Parameter	Type	Description	Handling Suggestions
			② If the version is too low, upgrade the controller or use an API version that is compatible.
-3	int	The return value parse fails, indicating that the received data format is incorrect or incomplete.	Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version compatibility, refer to the Version Description. ② If the version is too low, upgrade the controller or use an API version that is compatible.
-4	int	Timeout.	- Check Timeout Setting: In blocking mode, it supports configuring the timeout for waiting for the device to complete its motion. Ensure that the timeout is set longer than the device's motion time. - Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-5	int	The current in-position equipment verification fails, indicating the current in-position equipment is not a gripper.	- Detect Concurrent Control by Multiple Devices: Check if other devices are sending motion commands to the robot arm, including the motion of the robot arm, gripper, dexterous hand, and elevator. - Monitor Command Events in Real-Time: Register the callback function <code>rm_get_arm_event_call_back</code> : ① Capture device arrival events (such as motion completion, timeout, etc.). ② Determine the specific device type that triggered the event through the device parameter in the callback.

• Usage demo


```
from Robotic_Arm.rm_robot_interface import *

# Instantiate the RoboticArm class
arm = RoboticArm(rm_thread_mode_e.RM_TRIPLE_MODE_E)

# Create the robotic arm connection and print the connection ID
handle = arm.rm_create_robot_arm("192.168.1.18", 8080)
print(handle.id)

print(arm.rm_set_gripper_pick(500, 200, True, 10))

arm.rm_delete_robot_arm()
```

Gripper continuous force-controlled grasping `rm_set_gripper_pick_on()`

- Method prototype:

```
rm_set_gripper_pick_on(self, speed: int, force: int, block: bool, timeout: int) -> i
```

- Parameter description:

Parameter	Type	Description
<code>speed</code>	<code>int</code>	Grasping speed of the gripper, range: 1–1,000, without a unit of measurement.
<code>force</code>	<code>int</code>	Force control threshold, range: 50–1,000, without a unit of measurement.
<code>block</code>	<code>bool</code>	true: blocking mode, where the system waits for the controller to return the gripper in-position command; false: non-blocking mode, where no gripper in-position command is received.
<code>timeout</code>	<code>int</code>	Blocking mode: set the timeout period for waiting for the in-position gripper, in s; non-blocking mode: 0: immediately return after sending the command; other values: return after receiving the successful setting command.

- **Return value:**
State codes executed by functions:

Parameter	Type	Description	Handling Suggestions
0	int	Success.	-
1	int	The controller returns false, indicating that the parameters are sent incorrectly or the robotic arm state is wrong.	- Validate JSON Command: ① Enable DEBUG logs for the API to capture the raw JSON data. ② Check JSON syntax: Ensure correct formatting of parentheses, quotes, commas, etc. (You can use a JSON validation tool). ③ Verify against the API documentation that parameter names, data types, and value ranges comply with the specifications. ④ After fixing the issues, resend the command and check if the controller returns a normal status code and business data. - Check Robot Arm Status: ① Check for real-time error messages in the robot arm controller or logs (such as hardware failures, over-limit conditions), and reset, calibrate, or troubleshoot hardware issues according to the prompts. ② After fixing the issues, resend the command and check if the controller returns a normal status code and business data.
-1	int	The data transmission fails, indicating that a problem occurs during the communication.	Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-2	int	The data reception fails, indicating that a problem occurs during the communication, or	- Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal. - Verify Version Compatibility:

Parameter	Type	Description	Handling Suggestions
		the controller has a return timeout.	① Check if the controller firmware version supports the current API functions. For specific version compatibility, refer to the Version Description. ② If the version is too low, upgrade the controller or use an API version that is compatible.
-3	int	The return value parse fails, indicating that the received data format is incorrect or incomplete.	Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version compatibility, refer to the Version Description. ② If the version is too low, upgrade the controller or use an API version that is compatible.
-4	int	Timeout.	- Check Timeout Setting: In blocking mode, it supports configuring the timeout for waiting for the device to complete its motion. Ensure that the timeout is set longer than the device's motion time. - Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.
-5	int	The current in-position equipment verification fails, indicating the current in-position equipment is not a gripper.	- Detect Concurrent Control by Multiple Devices: Check if other devices are sending motion commands to the robot arm, including the motion of the robot arm, gripper, dexterous hand, and elevator. - Monitor Command Events in Real-Time: Register the callback function <code>rm_get_arm_event_call_back</code> : ① Capture device arrival events (such as motion completion, timeout, etc.). ② Determine the specific device type that

Parameter	Type	Description	Handling Suggestions
			triggered the event through the device parameter in the callback.

• Usage demo

python

```
from Robotic_Arm.rm_robot_interface import *

# Instantiate the RoboticArm class
arm = RoboticArm(rm_thread_mode_e.RM_TRIPLE_MODE_E)

# Create the robotic arm connection and print the connection ID
handle = arm.rm_create_robot_arm("192.168.1.18", 8080)
print(handle.id)

print(arm.rm_set_gripper_pick_on(500, 200, True, 10))

arm.rm_delete_robot_arm()
```



Set gripper to a specified position

`rm_set_gripper_position()`

• Method prototype:

python

```
rm_set_gripper_position(self, position: int, block: bool, timeout: int) -> int:
```

• Parameter description:

Parameter	Type	Description
<code>position</code>	<code>int</code>	Opening position of the gripper, range: 1–1,000, without a unit of measurement
<code>block</code>	<code>bool</code>	true: blocking mode, where the system waits for the controller to return the gripper in-position command; false: non-blocking mode, where no gripper in-position command is received.

Parameter	Type	Description
timeout	int	Blocking mode: set the timeout period for waiting for the in-position gripper, in s; non-blocking mode: 0: immediately return after sending the command; other values: return after receiving the successful setting command.

- Return value:
State codes executed by functions:

Parameter	Type	Description	Handling Suggestions
0	int	Success.	-
1	int	The controller returns false, indicating that the parameters are sent incorrectly or the robotic arm state is wrong.	- Validate JSON Command: ① Enable DEBUG logs for the API to capture the raw JSON data. ② Check JSON syntax: Ensure correct formatting of parentheses, quotes, commas, etc. (You can use a JSON validation tool). ③ Verify against the API documentation that parameter names, data types, and value ranges comply with the specifications. ④ After fixing the issues, resend the command and check if the controller returns a normal status code and business data. - Check Robot Arm Status: ① Check for real-time error messages in the robot arm controller or logs (such as hardware failures, over-limit conditions), and reset, calibrate, or troubleshoot hardware issues according to the prompts. ② After fixing the issues, resend the command and check if the controller returns a normal status code and business data.
-1	int	The data transmission fails, indicating that a	Check Network Connectivity: Use tools like ping/telnet to check if the

Parameter	Type	Description	Handling Suggestions
		problem occurs during the communication.	communication link with the controller is normal.
-2	int	The data reception fails, indicating that a problem occurs during the communication, or the controller has a return timeout.	- Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal. - Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version compatibility, refer to the Version Description. ② If the version is too low, upgrade the controller or use an API version that is compatible.
-3	int	The return value parse fails, indicating that the received data format is incorrect or incomplete.	Verify Version Compatibility: ① Check if the controller firmware version supports the current API functions. For specific version compatibility, refer to the Version Description. ② If the version is too low, upgrade the controller or use an API version that is compatible.
-4	int	Timeout.	- Check Timeout Setting: In blocking mode, it supports configuring the timeout for waiting for the device to complete its motion. Ensure that the timeout is set longer than the device's motion time. - Check Network Connectivity: Use tools like ping/telnet to check if the communication link with the controller is normal.

Parameter	Type	Description	Handling Suggestions
-5	int	The current in-position equipment verification fails, indicating the current in-position equipment is not a gripper.	- Detect Concurrent Control by Multiple Devices: Check if other devices are sending motion commands to the robot arm, including the motion of the robot arm, gripper, dexterous hand, and elevator. - Monitor Command Events in Real-Time: Register the callback function <code>rm_get_arm_event_call_back</code> : ① Capture device arrival events (such as motion completion, timeout, etc.). ② Determine the specific device type that triggered the event through the device parameter in the callback.

• Usage demo

python

```
from Robotic_Arm.rm_robot_interface import *

# Instantiate the RoboticArm class
arm = RoboticArm(rm_thread_mode_e.RM_TRIPLE_MODE_E)

# Create the robotic arm connection and print the connection ID
handle = arm.rm_create_robot_arm("192.168.1.18", 8080)
print(handle.id)

print(arm.rm_set_gripper_position(500, True, 10))

arm.rm_delete_robot_arm()
```

 Query gripper state `rm_get_gripper_state()`

• Method prototype:

python

```
rm_get_gripper_state(self) -> tuple[int, dict[str, any]]:
```

• **Return value:**

`tuple[int,dict[str, any]]` : a tuple containing two elements.

1. int: state codes executed by functions.

0 represents success. For other error codes, please refer to the [API2 Error Codes](#).

2. Gripper state information

Parameter	Type	Description
<code>rm_gripper_state_t</code>	<code>dict[str, any]</code>	Dictionary of gripper state information, key: field name of rm_gripper_state_t

• **Usage demo**

python

```
from Robotic_Arm.rm_robot_interface import *

# Instantiate the RoboticArm class
arm = RoboticArm(rm_thread_mode_e.RM_TRIPLE_MODE_E)

# Create the robotic arm connection and print the connection ID
handle = arm.rm_create_robot_arm("192.168.1.18", 8080)
print(handle.id)

print(arm.rm_get_gripper_state())

arm.rm_delete_robot_arm()
```

Last Update Time: 2025/4/1 17:49:08

Previous

[Global Waypoint Management](#)

Next

[Five-Fingered Dexterous Hand Configuration](#)